# ASSIGNMENT 5:

Mansha Bhatnagar
1024150192
2O23

1. Write a simple base class, then a derived class and use objects of both of them in the main function. It will be a simple illustration of inheritance.

```cpp
#include <iostream>
using namespace std;

class Base {
public:
    void showBase() {
        cout << "This is the Base class" << endl;
    }
};

class Derived : public Base {
public:
    void showDerived() {
        cout << "This is the Derived class" << endl;
    }
};

int main() {
    Base b;
    Derived d;
    b.showBase();
    d.showBase();
    d.showDerived();
    return 0;
}
```

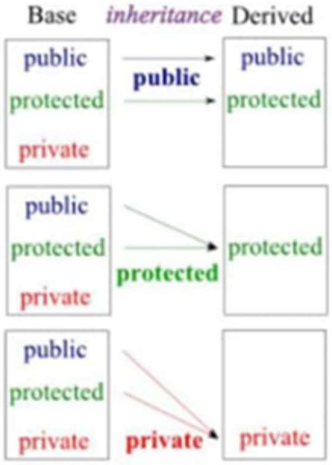
```
PS C:\Users\HP> cd "C:\Users\HP\App
++ tempCodeRunnerFile.cpp -o tempCo
mpCodeRunnerFile }
This is the Base class
This is the Base class
This is the Derived class
PS C:\Users\HP\AppData\Local\Temp>
```

2. Practice protected access specifier in inheritance. In the base class declare a variable which is protected and access it in the derived class.

```cpp
#include <iostream>
using namespace std;

class Parent {
protected:
    int secretCode = 1234;
};

class Child : public Parent {
public:
    void display() {
        cout << "Accessing protected member: " << secretCode << endl;
    }
};

int main() {
    Child c;
    c.display();
    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppData\Lo
++ tempCodeRunnerFile.cpp -o tempCodeRunne
mpCodeRunnerFile }
This is the Base class
This is the Base class
This is the Derived class
PS C:\Users\HP\AppData\Local\Temp> cd "C:\
emp\" ; if ($?) { g++ tempCodeRunnerFile.c
 } ; if ($?) { .\tempCodeRunnerFile }
Accessing protected member: 1234
PS C:\Users\HP\AppData\Local\Temp>
```

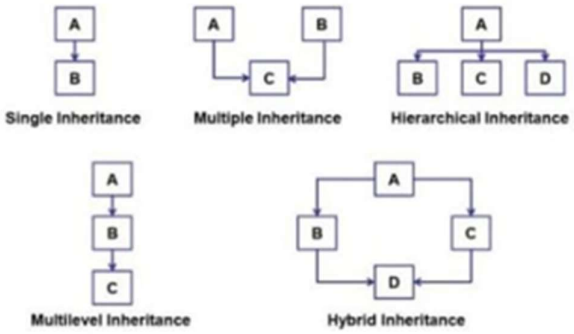
3. Most of the time we use public mode of inheritance, for example *class Derived: public Base{};* Try protected and private access modifiers to understand the difference of various modes of inheritance.




```cpp
#include <iostream>
using namespace std;

class A {
public: int x = 10;
protected: int y = 20;
private: int z = 30;
};

class B : public A {
    void test() { cout << x << y; }
};

class C : protected A {
    void test() { cout << x << y; }
};

class D : private A {
    void test() { cout << x << y; }
};

int main() {
    B objB;
    cout << "Public x: " << objB.x << endl;
    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppDa
++ tempCodeRunnerFile.cpp -o tempCode
mpCodeRunnerFile }
This is the Base class
This is the Base class
This is the Derived class
PS C:\Users\HP\AppData\Local\Temp> co
emp\" ; if ($?) { g++ tempCodeRunnerF
 } ; if ($?) { .\tempCodeRunnerFile }
Accessing protected member: 1234
PS C:\Users\HP\AppData\Local\Temp> co
emp\" ; if ($?) { g++ tempCodeRunnerF
 } ; if ($?) { .\tempCodeRunnerFile }
Public x: 10
PS C:\Users\HP\AppData\Local\Temp>
```

4. Various types of inheritances are as shown below. Write small C++ codes for each inheritance type.



```cpp
#include <iostream>
using namespace std;

class A { public: void msg() { cout << "Class A" << endl; } };
class B : public A {};
class C : public A {};
class D : public B, public C {};

int main() {
    B s; s.msg();
    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppData
++ tempCodeRunnerFile.cpp -o tempCodeRu
mpCodeRunnerFile }
Class A
PS C:\Users\HP\AppData\Local\Temp>
```

5. How can you use constructors and destructors in C++ inheritance? Write a program to illustrate.

```cpp
#include <iostream>
using namespace std;

class Parent {
public:
    Parent() { cout << "Parent Constructor" << endl; }
    ~Parent() { cout << "Parent Destructor" << endl; }
};

class Child : public Parent {
public:
    Child() { cout << "Child Constructor" << endl; }
    ~Child() { cout << "Child Destructor" << endl; }
};

int main() {
    Child obj;
    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppDat
++ tempCodeRunnerFile.cpp -o tempCodeR
mpCodeRunnerFile }
Class A
PS C:\Users\HP\AppData\Local\Temp> cd
emp\" ; if ($?) { g++ tempCodeRunnerFi
 } ; if ($?) { .\tempCodeRunnerFile }
Parent Constructor
Child Constructor
Child Destructor
Parent Destructor
PS C:\Users\HP\AppData\Local\Temp>
```

6. Implement a base class Book with attributes title, author, and price. Then, create a derived class Textbook that inherits from Book and adds a new attribute subject. Demonstrate how single inheritance is used to manage the data for general books and textbooks.

```cpp
#include <iostream>
#include <string>
using namespace std;

class Book {
protected:
    string title, author;
    float price;
public:
    void setBook(string t, string a, float p) {
        title = t; author = a; price = p;
    }
};

class Textbook : public Book {
    string subject;
public:
    void setTextbook(string t, string a, float p, string s) {
        setBook(t, a, p);
        subject = s;
    }
    void show() {
        cout << title << " by " << author << " [" << subject << "] Price
    }
};

int main() {
    Textbook t;
    t.setTextbook("C++ OOP", "E Balaguruswamy", 450.0, "Programming");
    t.show();
    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppData\Local\Temp\" ; if (
++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?)
mpCodeRunnerFile }
Class A
PS C:\Users\HP\AppData\Local\Temp> cd "C:\Users\HP\AppData\
emp\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRun
 } ; if ($?) { .\tempCodeRunnerFile }
Parent Constructor
Child Constructor
Child Destructor
Parent Destructor
PS C:\Users\HP\AppData\Local\Temp> cd "C:\Users\HP\AppData\
emp\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRun
 } ; if ($?) { .\tempCodeRunnerFile }
C++ OOP by E Balaguruswamy [Programming] Price: 450
PS C:\Users\HP\AppData\Local\Temp>
```

7. A software company is creating a program to simulate a car's dashboard. The dashboard needs to display speed, fuel level, and temperature. The speed is controlled by a Speedometer class, the fuel level by a FuelGauge class, and the temperature by a Thermometer class. Implement the three classes: Speedometer, FuelGauge, and Thermometer, each with relevant attributes and methods. Then, create a CarDashboard class that inherits from all three classes to display the combined information on the dashboard. Demonstrate how multiple inheritance is used to build this class.

```cpp
#include <iostream>
using namespace std;

class Speedometer {
protected: float speed = 80.5;
};

class FuelGauge {
protected: float fuel = 45.0;
};

class Thermometer {
protected: float temp = 98.6;
};

class CarDashboard : public Speedometer, public FuelGauge, publi Therm
public:
    void display() {
        cout << "Speed: " << speed << " km/h" << endl;
        cout << "Fuel: " << fuel << " L" << endl;
        cout << "Temp: " << temp << " F" << endl;
    }
};

int main() {
    CarDashboard dash;
    dash.display();
    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppData\Loc
++ tempCodeRunnerFile.cpp -o tempCodeRunner
mpCodeRunnerFile }
Speed: 80.5 km/h
Fuel: 45 L
Temp: 98.6 F
PS C:\Users\HP\AppData\Local\Temp>
```

8. You are tasked with creating a system for a library that tracks different types of users. The system needs to handle general user information such as name, ID, and contact details. There are two specific types of users: Student and Teacher. Each type of user has additional attributes, such as grade level for students and department for teachers. Implement a base class LibraryUser with general attributes. Then, create two derived classes Student and Teacher that inherit from LibraryUser and add their own specific attributes. Demonstrate how hierarchical inheritance is applied in this scenario.

```cpp
#include <iostream>
#include <string>
using namespace std;

class LibraryUser {
protected:
    string name;
    int id;
public:
    void setBasic(string n, int i) { name = n; id = i; }
};

class Student : public LibraryUser {
    int grade;
public:
    void setStudent(string n, int i, int g) { setBasic(n, i); grade = g;
    void show() { cout << "Student: " << name << " Grade: " << grade <<
};

class Teacher : public LibraryUser {
    string dept;
public:
    void setTeacher(string n, int i, string d) { setBasic(n, i); dept =
    void show() { cout << "Teacher: " << name << " Dept: " << dept << e
};

int main() {
    Student s; Teacher t;
    s.setStudent("Alice", 101, 10);
    t.setTeacher("Dr. Smith", 501, "CS");
    s.show(); t.show();
    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppData\Local
++ tempCodeRunnerFile.cpp -o tempCodeRunnerF
mpCodeRunnerFile }
Speed: 80.5 km/h
Fuel: 45 L
Temp: 98.6 F
PS C:\Users\HP\AppData\Local\Temp> cd "C:\Us
emp\" ; if ($?) { g++ tempCodeRunnerFile.cpp
 } ; if ($?) { .\tempCodeRunnerFile }
Student: Alice Grade: 10
Teacher: Dr. Smith Dept: CS
PS C:\Users\HP\AppData\Local\Temp>
```

9. A logistics company needs a software system to manage its vehicle fleet. All vehicles share common attributes like make, model, and year. Trucks have additional attributes like load_capacity. Furthermore, refrigerated trucks have a special attribute called temperature_control. Implement a base class Vehicle with common attributes. Then, create a derived class Truck that adds the load_capacity attribute. Finally, create another derived class RefrigeratedTruck that inherits from Truck and adds the temperature_control attribute. Demonstrate how multilevel inheritance works in this case.

```cpp
#include <iostream>
#include <string>
using namespace std;

class Vehicle {
protected:
    string make = "Tata", model = "Prima";
};

class Truck : public Vehicle {
protected:
    int load_capacity = 25000;
};

class RefrigeratedTruck : public Truck {
    float temp_control = -18.0;
public:
    void show() {
        cout << make << " " << model << " Cap: " << load_capacity << "kg
    }
};

int main() {
    RefrigeratedTruck rt;
    rt.show();
    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppData\Loca
++ tempCodeRunnerFile.cpp -o tempCodeRunnerF
mpCodeRunnerFile }
Tata Prima Cap: 25000kg Temp: -18C
PS C:\Users\HP\AppData\Local\Temp>
```

10. You are developing a software system for an academic institution. The institution has various roles like Person, Staff, and Student. A Person has general attributes like name and address. Staff members, who are a type of Person, have additional attributes like employee_id and department. Student, another type of Person, has attributes like student_id and grade. Some Staff members are also Students (e.g., teaching assistants) and need to inherit from both classes. Implement a base class Person with general attributes. Then, create derived classes Staff and Student that inherit from Person and add their specific attributes. Finally, create a TeachingAssistant class that inherits from both Staff and Student. Demonstrate how hybrid inheritance is applied and managed in this scenario.

```cpp
using namespace std;

class Person {
protected:
    string name = "John", address = "Patiala";
};

class Staff : virtual public Person {
protected:
    int emp_id = 1234;
    string dept = "Admin";
};

class Student : virtual public Person {
protected:
    int student_id = 9876;
    int grade = 12;
};

class TeachingAssistant : public Staff, public Student {
public:
    void display() {
        cout << "Name: " << name << endl;
        cout << "Emp ID: " << emp_id << endl;
        cout << "Student ID: " << student_id << endl;
    }
};

int main() {
    TeachingAssistant ta;
    ta.display();
    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppData\Loca
++ tempCodeRunnerFile.cpp -o tempCodeRunnerF
mpCodeRunnerFile }
Tata Prima Cap: 25000kg Temp: -18C
PS C:\Users\HP\AppData\Local\Temp> cd "C:\Us
emp\" ; if ($?) { g++ tempCodeRunnerFile.cpp
 } ; if ($?) { .\tempCodeRunnerFile }
Name: John
Emp ID: 1234
Student ID: 9876
PS C:\Users\HP\AppData\Local\Temp>
```